



Instituto Tecnológico de Buenos Aires

Departamento de Ciencias Exactas y Naturales

93.24 - PROBABILIDAD Y ESTADÍSTICA

EAC 2

ESTIMACIÓN PUNTAL E INTERVALOS ESTADÍSTICOS

GRUPO N° 19

Santiago Feldman - 62012

Theo Shlamovitz - 62087

Emilio Pablo Neme - 62601

Gonzalo Candisiano - 62616

Alberto Bendayan - 62786

Tobias Ves Losada - 63342

PROFESORES

Pablo Ignacio Fierens

Lucio José Pantazis

Alejandro Hayes

Fecha de entrega: 4 de junio de 2023

NOTA:

Fecha de corrección:

Observaciones:

Índice

1. Enunciado	1
2. Resolución	2
2.1. Ejercicio 1	2
2.2. Ejercicio 2	4
A. Código implementado	I
Bibliografía	III

Lista de figuras

2.1. Polígonos de frecuencias relativas de la variable aleatoria z_n	4
--	---

1. Enunciado

Señal Sinusoidal A

Considere una señal sinusoidal

$$x_n = \alpha \cos(2\pi f_0 t_n) + w_n \quad (1)$$

donde $f_0 = 10^6$, $w_n \sim \mathcal{N}(0, \sigma)$ i.i.d., $n=0,1,\dots,999$. Los parámetros α y σ son desconocidos.

El archivo fouriera1.txt contiene dos columnas: la primera con valores de t_n y la segunda con valores de x_n .

1. Máxima verosimilitud:
 - (a) Encuentre estimadores de máxima verosimilitud para los parámetros desconocidos.
 - (b) Calcule la estimación puntual para los datos del archivo.
 - (c) Compare el polígono de frecuencias relativas basado en los datos con las probabilidades para cada bin obtenidas a partir de la distribución teórica de probabilidad con el parámetro estimado.
2. En un sistema de comunicación, se recibe una señal como la anterior, con $\sigma = 1$ y $\alpha = 0,1$. Se comete un error en la detección de la señal cuando la estimación de α da menor a 0.
 - (a) Estime la probabilidad de cometer un error.
 - (b) Encuentre un intervalo de confianza al 95 % para dicha probabilidad.

2. Resolución

2.1 Ejercicio 1

De la Ecuación 1, se tiene que las variables aleatorias x_n tienen una distribución normal de media $\mu = \alpha \cos(2\pi f_0 t_n)$ y varianza igual a la de las variables aleatorias w_n (σ^2):

$$x_n \sim \mathcal{N}(\alpha \cos(2\pi f_0 t_n), \sigma^2) \quad (2)$$

Por lo tanto, la función densidad de probabilidad para x_n es:

$$f(x_n; \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left\{-\frac{(x_n - \alpha \cos(2\pi f_0 t_n))^2}{2\sigma^2}\right\} \quad (3)$$

Para estimar los parámetros desconocidos α y σ utilizando el método de máxima verosimilitud, se construye la función de verosimilitud $L(\mu, \sigma^2)$ como la productoria de las densidades de probabilidad para cada x_n , $n=0, \dots, N-1$. Nótese que, al tener 1000 valores de x_n y t_n en el archivo .txt y como w_n son 1000 variables aleatorias normales independientes e idénticamente distribuidas, $N=1000$.

$$L(\mu, \sigma^2) = f(x_0; \mu, \sigma^2) f(x_1; \mu, \sigma^2) \cdots f(x_{N-1}; \mu, \sigma^2) \quad (4)$$

Reemplazando la Ecuación 3 en la Ecuación 4:

$$L(\mu, \sigma^2) = (2\pi\sigma^2)^{-\frac{N}{2}} \exp\left\{-\frac{1}{2\sigma^2} \sum_{i=0}^{N-1} (x_i - \mu)^2\right\} \quad (5)$$

Para hallar los parámetros desconocidos, se aplica el logaritmo natural a ambos lados de la Ecuación 5:

$$\ln[L(\mu, \sigma^2)] = -\frac{N}{2} \ln(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{i=0}^{N-1} (x_i - \alpha \cos(2\pi f_0 t_i))^2 \quad (6)$$

Al ser el logaritmo una función creciente, si se maximiza $\ln[L(\mu, \sigma^2)]$ también se maximiza la función de verosimilitud propiamente. Esta transformación es conveniente porque el logaritmo convierte los productos en sumas, con las cuales es más sencillo operar.

Para hallar los estimadores de los parámetros desconocidos, se deriva el logaritmo de la verosimilitud con respecto al parámetro pertinente.

$$\frac{\partial \ln[L(\alpha \cos(2\pi f_0 t_i), \sigma^2)]}{\partial \alpha} = \frac{1}{\sigma^2} \left[\sum_{i=0}^{N-1} x_i \cos(2\pi f_0 t_i) - \alpha \sum_{i=1}^{N-1} \cos^2(2\pi f_0 t_i) \right] \quad (7)$$

$$\frac{\partial \ln[L(\mu, \sigma^2)]}{\partial \sigma^2} = -\frac{N}{2\sigma^2} + \frac{1}{2(\sigma^2)^2} \sum_{i=0}^{N-1} (x_i - \mu)^2 \quad (8)$$

Luego, se igualan las derivadas a 0 y se despejan los parámetros para obtener sus estimadores de máxima verosimilitud. De esa manera, los valores que se obtienen son los que más concuerdan con los datos proporcionados. Téngase en cuenta que para estimar σ primero se debe estimar α .

$$\hat{\alpha} = \frac{\sum_{i=0}^{N-1} x_i \cos(2\pi f_0 t_i)}{\sum_{i=0}^{N-1} \cos^2(2\pi f_0 t_i)} \quad (9)$$

$$\hat{\sigma} = \sqrt{\frac{1}{N} \sum_{i=0}^{N-1} (x_i - \hat{\alpha} \cos(2\pi f_0 t_i))^2} \quad (10)$$

Implementando las expresiones obtenidas en *Python*, se tiene que $\alpha = 0,04878952064117914$ y $\sigma = 1,0167140046432663$.

Para comparar el polígono de frecuencias relativas basado en los datos con las probabilidades para cada bin obtenidas a partir de la distribución teórica de probabilidad con el parámetro estimado, se normalizó x_n como:

$$z_n = \frac{x_n - \hat{\alpha} \cos(2\pi f_0 t_n)}{\hat{\sigma}} \quad (11)$$

Si los parámetros estuvieran perfectamente estimados, $z_n \sim \mathcal{N}(0, 1)$.

Se obtuvieron 1000 valores de z_n a partir de los valores de x_n y t_n proporcionados en el archivo de texto y se graficó el polígono de frecuencias relativas respecto a esos datos. El polígono de frecuencias se graficó tomando los puntos medios de los bins del histograma de z_n . Luego, se calcularon las probabilidades para cada bin a_i . Dicha probabilidad viene dada por:

$$p_{(a_i, a_{i+1}]} = \int_{a_i}^{a_{i+1}} f_{z_n}(z) = \Phi(a_{i+1}) - \Phi(a_i) \quad (12)$$

Por lo tanto, las probabilidades para cada bin se calcularon a partir de la función de distribución acumulada (FDA) de la normal estándar.

Se graficó el polígono de frecuencias obtenido a partir de los cálculos junto con el polígono obtenido a partir de los datos. Se utilizaron 14 bins para ambos polígonos. En la Figura 2.1 se puede notar que el polígono basado en las probabilidades y el obtenido a partir de los datos son muy similares, lo que confirma que se realizaron buenas estimaciones para α y σ .

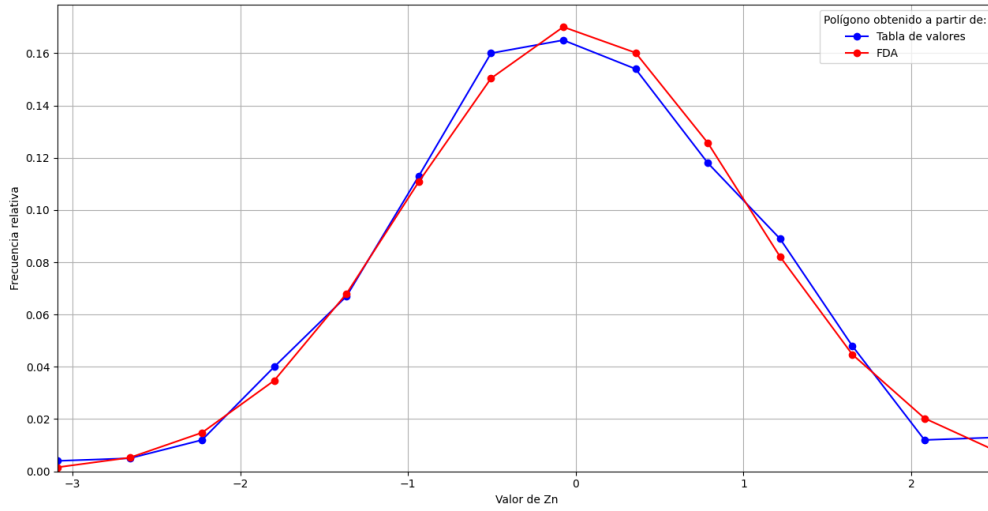


Figura 2.1: Polígonos de frecuencias relativas de la variable aleatoria z_n

2.2 Ejercicio 2

Para calcular la probabilidad de cometer un error en la detección de la señal, se calcularon 1000 estimaciones de α utilizando máxima verosimilitud. Para cada estimación, se generaron 1000 valores de x_n con la Ecuación 1, utilizando 1000 normales estándar $w_n \sim \mathcal{N}(0, 1)$ previamente generadas, los 1000 valores de t_n del archivo de texto y $\alpha = 0, 1$. Luego, la probabilidad de cometer un error se calculó utilizando la regla de Laplace:

$$p = \frac{\# \text{ casos favorables}}{\# \text{ casos totales}} = \frac{\# \text{ veces en las que se detectó } \alpha < 0}{1000} \quad (13)$$

Al generar aleatoriamente las normales estándar, el resultado de p cambia con cada compilación. Para cierta compilación, se obtiene un valor de $p = 0,014$.

Para calcular el intervalo de confianza bilateral de la probabilidad p , como el tamaño de la muestra es grande (1000), se aproximó la distribución del estimador $\hat{p}$ por una normal. De esa manera, los intervalos de confianza se calcularon utilizando la siguiente expresión:

$$IC_{95\%}(p) = \left[\hat{p} - z_{0,975} \sqrt{\frac{\hat{p}(1 - \hat{p})}{1000}}; \hat{p} + z_{0,975} \sqrt{\frac{\hat{p}(1 - \hat{p})}{1000}} \right] \quad (14)$$

Por lo tanto, para la compilación en la que $p = 0,014$, se tiene que el intervalo de confianza al 95 % está dado por $IC_{95\%}(p = 0,014) \approx [0,00672; 0,02128]$ ($p = 0,014 \pm 0,00728$).

A. Código implementado

```

1 import matplotlib.pyplot as plt
2 import numpy as np
3 from math import sqrt
4 from scipy.stats import norm
5
6 # Path donde está guardado el archivo fouriera1.txt (cambiar pertinentemente)
7 file_path = "../fouriera1.txt"
8
9 fo = 1E6
10
11 # Estimación del parámetro alfa de la variable aleatoria Xn utilizando máxima verosimilitud
12 def normal_alpha_MLE(X,T):
13     s1 = 0
14     s2 = 0
15     for i in range(len(X)):
16         s1 += X[i]*np.cos(2*np.pi*fo*T[i])
17         s2 += (np.cos(2*np.pi*fo*T[i]))**2
18     return s1/s2
19
20 # Estimación del parámetro sigma de la variable aleatoria Wn utilizando máxima verosimilitud
21 def normal_sigma_MLE(X, T, alpha):
22     s = 0
23     for i in range(len(X)):
24         s += (X[i]-alpha*np.cos(2*np.pi*fo*T[i]))**2
25     return (sqrt(s/len(X)))
26
27 # Guardo los valores de Xn y tn del archivo fouriera1.txt
28 xn_values = []
29 tn_values = []
30
31 with open(file_path, 'r') as file:
32     # Se descartan las primeras 1002 líneas
33     for _ in range(1002):
34         next(file)
35
36     for line in file:
37         values = line.split()
38         tn = float(values[0])
39         xn = float(values[1])
40
41         xn_values.append(xn)
42         tn_values.append(tn)
43
44 # Estimación de alfa y sigma
45 alpha = normal_alpha_MLE(xn_values, tn_values)
46 sigma = normal_sigma_MLE(xn_values, tn_values, alpha)
47
48 print("alpha:", alpha, "\nsigma:", sigma)
49
50 # Normalización de Xn a Zn
51 zn_values = []
52 for i in range(len(xn_values)):
53     zn_values.append((xn_values[i]-alpha*np.cos(2*np.pi*fo*tn_values[i]))/sigma)
54
55 # Polígono de frecuencia relativa de Zn
56 bin_number = 14
57 y,edges,_ = plt.hist(zn_values, bin_number, color='white',
58                     weights=np.ones_like(zn_values) / len(zn_values))
59 midpoints = 0.5*(edges[1:]+edges[:-1])
60

```



```

61 # Obtención del polígono de frecuencias a partir de la función de distribución acumulada de Zn
62 polygon = []
63 for i in range(len(edges)-1):
64     polygon.append(norm.cdf(edges[i+1]) - norm.cdf(edges[i]))
65
66 # Muestro los polígonos
67 plt.xlim((edges[0]+edges[1])/2, (edges[-1]+edges[-2])/2)
68 plt.plot(midpoints, y, color='b', label='Tabla de valores', marker='o')
69 plt.plot(midpoints, polygon, color='r', label='FDA', marker='o')
70 plt.ylabel('Frecuencia relativa')
71 plt.xlabel('Valor de Zn')
72 plt.legend(title='Polígono obtenido a partir de:')
73 plt.grid()
74 plt.show()
75
76 # Genero 1000 estimaciones de alpha y verifico cuántas son menores a cero
77 alpha = 0.1
78 num_errors = 0
79 num_iterations = 1000
80
81 for iteration in range(num_iterations):
82
83     wn_values = np.random.normal(0, 1, 1000)
84
85     for i in range(len(wn_values)):
86         xn_values[i] = alpha * np.cos(2 * np.pi * fo * tn_values[i]) + wn_values[i]
87
88     current_alpha = normal_alpha_MLE(xn_values, tn_values)
89
90     if current_alpha < 0:
91         num_errors += 1
92
93 error_probability = num_errors / num_iterations
94
95 print("p:", error_probability)
96
97 # Cálculo del intervalo de confianza
98 confidence_interval = norm.ppf(0.975) * sqrt((error_probability * (1 - error_probability)) / num_iterations)
99
100 upper_level = error_probability + confidence_interval
101 lower_level = error_probability - confidence_interval
102
103 print("IC:", confidence_interval,
104       "\nLímite superior:", upper_level, "\nLímite inferior:", lower_level)

```

Bibliografía

- [1] Devore, J. (2016). *Probabilidad y Estadística para Ingeniería y Ciencias* 9a. Ed. México D.F., Mexico : Cengage Learning Editores